Lab9: Containerising the Lab 7

Docker and Docker Compose

Name: Aditya Yadav | Roll No: 12340100 | Date: 28 October 2026

1. Objective

The goal of this lab was to containerise the Lab 7 Proximity Search API so that the whole stack starts with a single command. Each service has its own Dockerfile and one `docker-compose.yml` builds and runs everything. All work was done on my personal laptop, not on the allotted cluster systems.

Item	Value
Machine	MacBook Air (M1, 8 GB RAM), macOS 15.5, arm64
Docker engine	Docker Desktop 4.77.0 (Engine 29.5.3, linux/arm64 VM, 8 CPUs, ~3.8 GiB RAM)
Docker CLI / Compose	Docker CLI 29.7.2, Docker Compose 5.4.0 (both from Homebrew)
One-command start	`docker compose up -d --build`

2. What was containerised

On the cluster, Lab 7 ran as four Uvicorn processes (sys1 to sys4) managed by Supervisor, with Nginx on sys4 exposing port 3204 and forwarding `/IP/search/` to the API. I kept the same shape in Docker, so the stack has two services:

- **api**: the FastAPI/Uvicorn proximity search service (`app/main.py`). It loads `locations.csv` into memory, filters by category and circular radius, and ranks using Dijkstra traversal distance. It runs as replicas (2 by default) in place of the four cluster nodes.
- **gateway**: Nginx, the only service reachable from the host. It listens on port 3204 like on sys4 and load-balances requests across the API replicas.

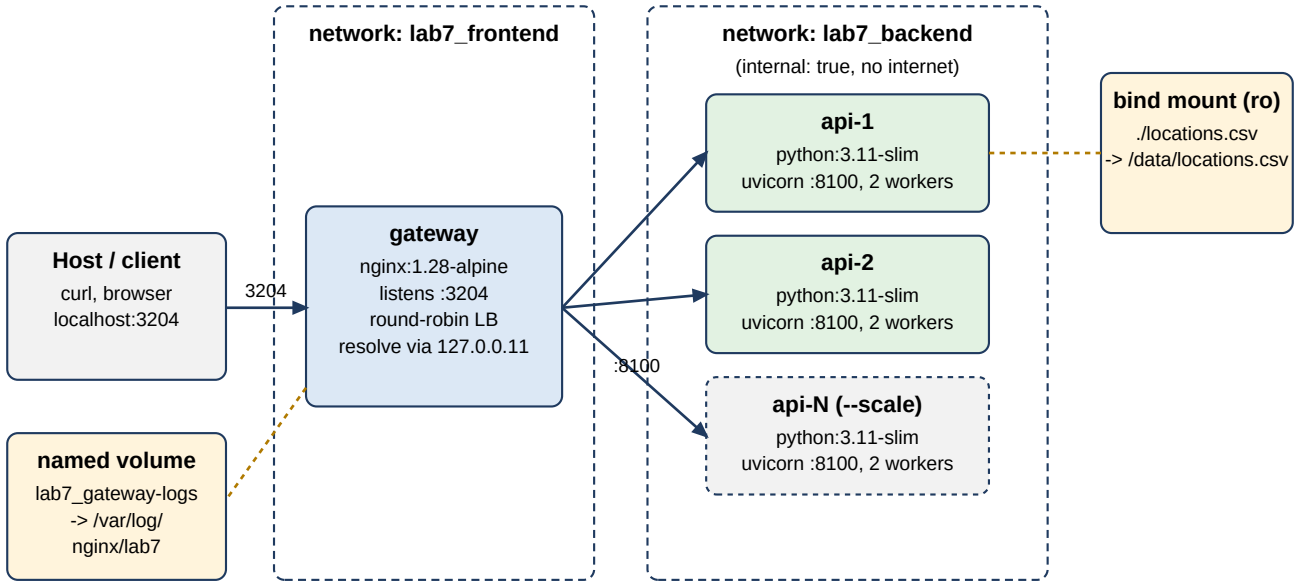


Figure 1: Stack layout. Only the gateway publishes a port; the API replicas sit on an internal network.

Files added or changed in `lab7/`:

File	Purpose
`Dockerfile`	API image (rewritten from the earlier single-stage draft)
`gateway/Dockerfile`	Gateway image based on nginx:1.28-alpine
`gateway/templates/default.conf.template`	Nginx config; environment variables are filled in at container start
`docker-compose.yml`	Builds both images and defines networks, volumes, health checks, replicas and limits
`.dockerignore`	Keeps reports, test files, caches and link.txt out of the API build context
`.env.example`	Documented defaults; copied to .env (which is git-ignored)

The full contents of these files are in the Appendix.

3. Docker design choices

3.1 Base images

Service	Base image	Why
api	`python:3.11-slim`	Debian slim with Python 3.11, the version Lab 7 was developed on. glibc means the prebuilt arm64/amd64 wheels for uvloop and httptools (from uvicorn[standard]) install without a compiler. Alpine would need musl builds or build tools. Final image: 190 MB.
gateway	`nginx:1.28-alpine`	Current stable Nginx line. 1.28 supports the `resolve` flag in an upstream block (open-source since 1.27.3), which I needed for scaling (Section 5, error 7). Alpine keeps it small at 61.4 MB, and the official image renders `*.template` files with envsubst at startup.

3.2 Ports

Port	Where	Decision
3204	gateway, published as `${GATEWAY_PORT:-3204}:3204`	Same public port as the cluster deployment, so the earlier URLs and curl commands still work. The host side can be changed in .env if 3204 is busy.
8100	api, `expose` only	Not published to the host. Only the gateway can reach it, over the backend network. All replicas can use the same port because each has its own network namespace, so the cluster's 8101 to 8104 split is no longer needed.

3.3 Networks

- `lab7_frontend` (bridge): holds the gateway and is where the published port is attached.
- `lab7_backend` (bridge, `internal: true`): holds the gateway and every API replica. Because it is internal, the API containers have no route to the internet or the host, which I checked (Figure 13). The gateway reaches replicas by the service name `api` through Docker's embedded DNS at 127.0.0.11.

3.4 Volumes

- **Bind mount (read-only)**: `${DATASET_FILE:-./locations.csv}` -> `/data/locations.csv`. A copy of the CSV is also built into the image so it works on its own, but the mount lets a different dataset (for example the 1,000,000-row evaluation file) be used without rebuilding.
- **Named volume** `lab7_gateway-logs` -> `/var/log/nginx/lab7`: Nginx access logs that survive `docker compose down` and container recreation (verified: 842 log lines before and after a down/up). Access logs also go to stdout, so `docker compose logs` still shows them.
- **tmpfs** `/tmp` on the API: the API container's root filesystem is `read_only: true`, and Starlette writes uploaded linkage files larger than 1 MB to /tmp. A 2.1 MB linkage upload worked with this setup.

3.5 Environment variables

Variable	Default	Used by	Effect
GATEWAY_PORT	3204	compose	Host port mapped to the gateway
API_PORT	8100	api, gateway	Port Uvicorn listens on; also written into the Nginx upstream
API_REPLICAS	2	compose	deploy.replicas for the api service
UVICORN_WORKERS	2	api	Worker processes per container (same as the cluster)
UVICORN_LIMIT_CONCURRENCY	64	api	Per-worker concurrency cap, kept from Lab 7
UVICORN_BACKLOG	2048	api	Accept backlog, kept from Lab 7
LOCATION_DATASET	/data/locations.csv	api	CSV path read by app/main.py
DATASET_FILE	./locations.csv	compose	Host file bind-mounted as the dataset
API_HOST	api	gateway	Upstream DNS name used in the Nginx template
PYTHONUNBUFFERED, PYTHONDONTWRITEBYTECODE	1	api	Logs appear immediately; no .pyc writes on a read-only filesystem

Defaults live in the compose file as `${VAR:-default}`, so the stack runs without an .env file. `.env.example` documents every override.

3.6 Other decisions

- **Health checks.** The API image checks `/health` using Python's urllib, because the slim image has no curl. The gateway checks its own `/gateway-health` with BusyBox wget. Compose starts the gateway only once the API is `service_healthy`.
- **Process supervision.** `restart: unless-stopped` replaces Supervisor's autorestart. `CMD` uses `exec uvicorn ...` so Uvicorn is PID 1 and receives SIGTERM directly for a graceful stop.
- **Least privilege.** The API runs as the system user `lab7` (uid 999) on a read-only root filesystem, with 1 CPU and 512 MB of memory per replica.
- **Build speed.** Dependencies are installed before the code is copied, using a BuildKit pip cache mount, so changing `main.py` does not reinstall packages. `.dockerignore` keeps the build context to about 300 kB.
- **Log rotation.** The json-file log driver is capped at 10 MB x 5 files, matching the Supervisor log rotation used on the cluster.
- **Load balancing.** Nginx re-resolves `api` every 10 s (`server api:8100 resolve`), so replicas added with `--scale` get traffic without restarting the gateway. Each response includes an `X-Upstream` header naming the replica that served it.

4. Steps followed

4.1 Install and verify Docker

Docker Desktop for Mac (Apple silicon) provides the Linux VM and engine. The `docker` CLI and the `docker-compose` plugin were installed with Homebrew (`brew install docker docker-compose`). I then checked that the client, Compose and the engine could all talk to each other, and pulled the two base images.

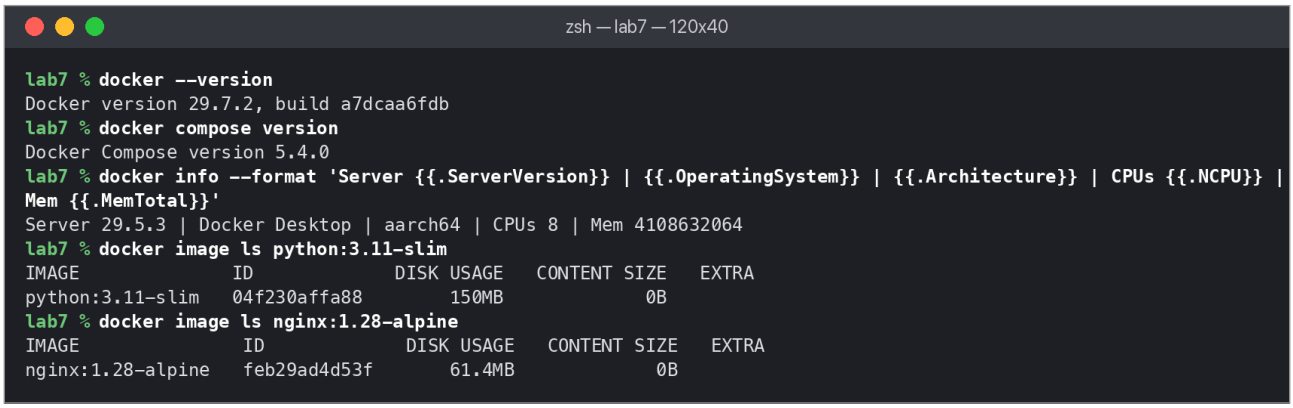


Figure 2: Docker CLI, Compose and engine versions, and the two base images present locally.

4.2 Prepare the project

I created `gateway/` with its Dockerfile and config template, rewrote the API `Dockerfile`, added `docker-compose.yml`, `.dockerignore` and `.env.example`, then ran `cp .env.example .env` and checked the compose file with `docker compose config`.

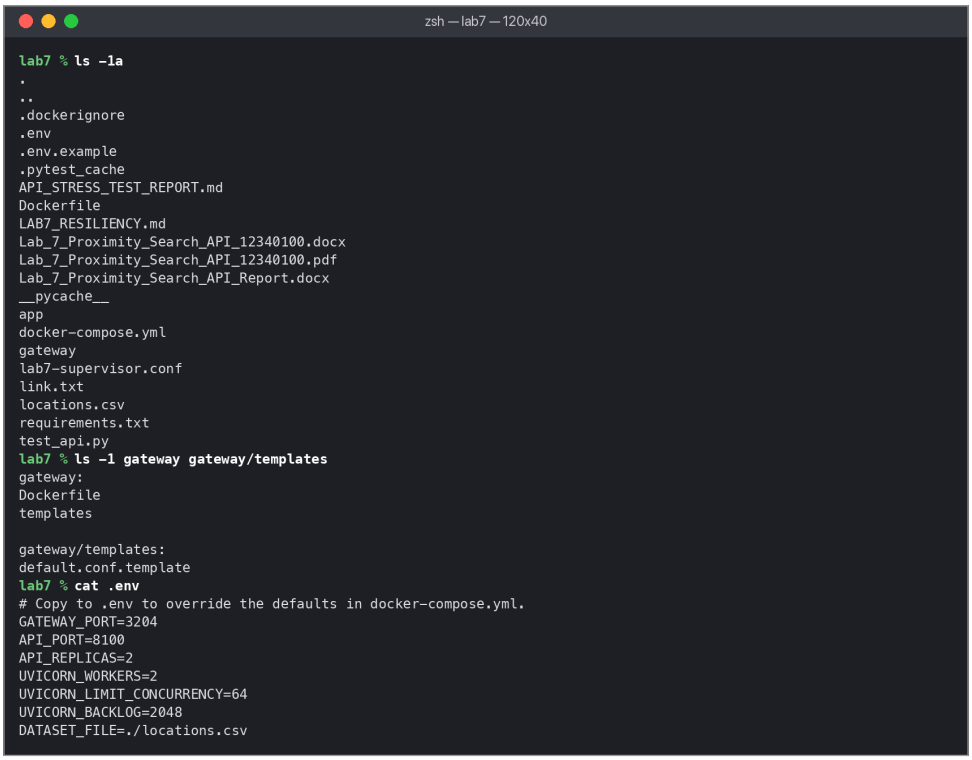


Figure 3: lab7 folder after adding the Docker files, and the .env values used.

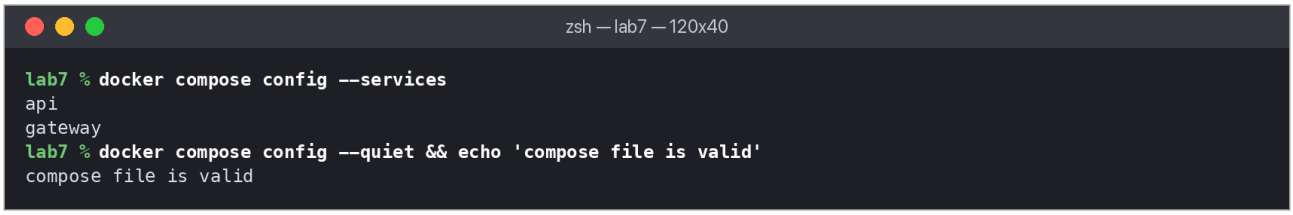


Figure 4: docker compose config confirms the two services and a valid file.

4.3 Build and run with one command

The first full build of the final Dockerfiles took 37 s, almost all of it the pip install. In the run below, every container, network and volume had been removed first, but the build layers were cached, so the full stack came up in 6.7 s. Compose builds both images, creates the two networks and the volume, starts the API replicas, waits for them to be healthy, and only then starts the gateway.

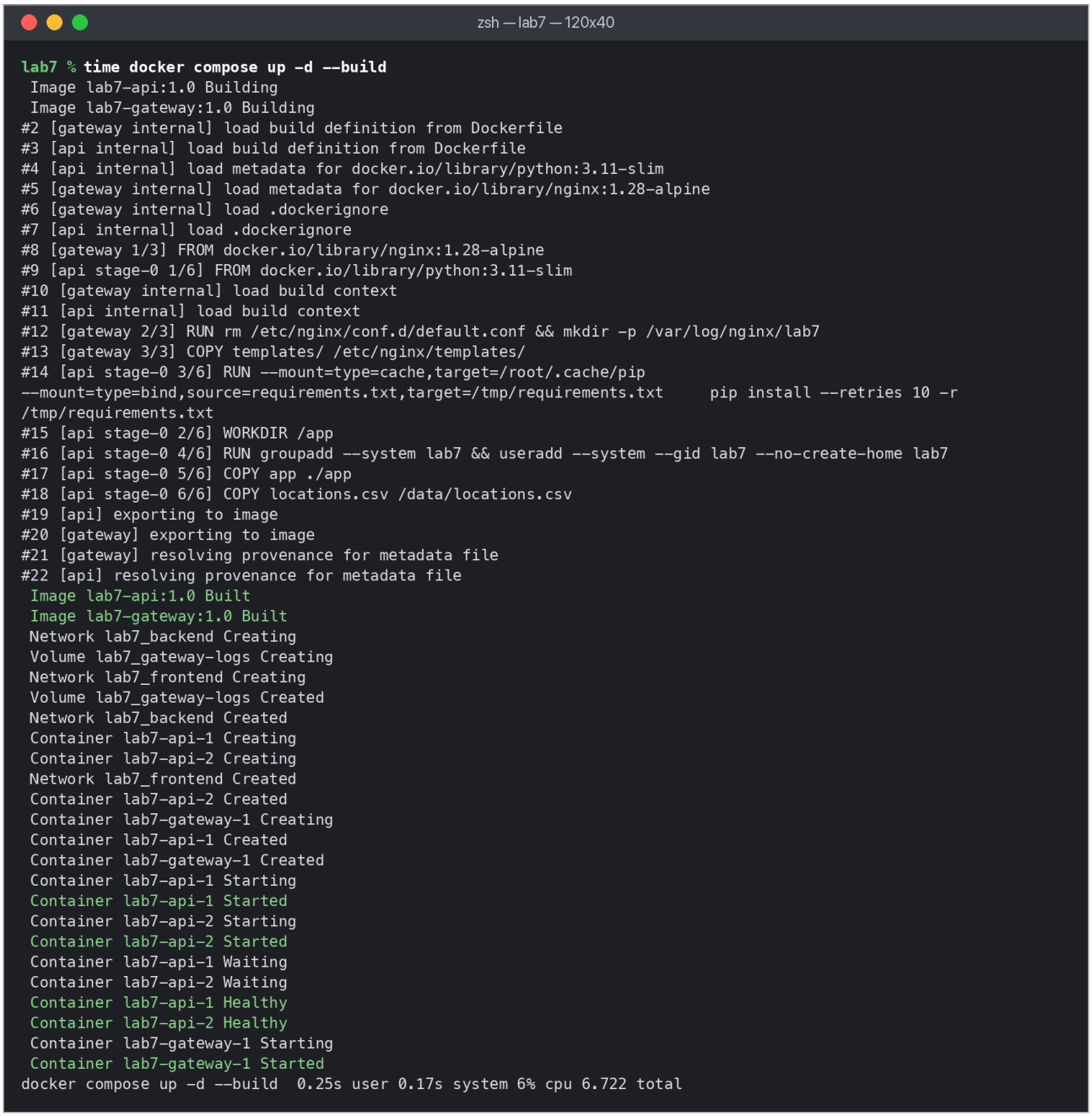


Figure 5: docker compose up -d --build: build steps (abridged) followed by network, volume and container creation.

```
zsh — lab7 — 120x40

lab7 % docker compose ps
NAME             IMAGE              COMMAND                  SERVICE   CREATED          STATUS
PORTS
lab7-api-1       lab7-api:1.0       "sh -c 'exec uvicorn…"   api       14 seconds ago   Up 14 seconds (healthy)
8100/tcp
lab7-api-2       lab7-api:1.0       "sh -c 'exec uvicorn…"   api       14 seconds ago   Up 13 seconds (healthy)
8100/tcp
lab7-gateway-1   lab7-gateway:1.0   "/docker-entrypoint.…"   gateway   14 seconds ago   Up 8 seconds (healthy)
0.0.0.0:3204->3204/tcp, [::]:3204->3204/tcp
lab7 % docker compose images
CONTAINER           REPOSITORY          TAG                 PLATFORM            IMAGE ID            SIZE
              CREATED
lab7-api-1          lab7-api            1.0                 linux/arm64         fde4d7394a53        190MB
              5 minutes ago
lab7-api-2          lab7-api            1.0                 linux/arm64         fde4d7394a53        190MB
              5 minutes ago
lab7-gateway-1      lab7-gateway        1.0                 linux/arm64         ac8dc9cdde4f        61.4MB
              6 minutes ago
```

Figure 6: All containers healthy. Only the gateway publishes a port (3204); the API shows 8100/tcp internally.

4.4 Test the running stack

Every request goes through the gateway on port 3204. The cross-node test query from the Lab 7 stress test returns the same ten IDs as on the cluster (`["3562","3565",...,"4362"]`). The POST request with the supplied 14,800-line `link.txt` works, invalid input is still rejected with 422, and port 8100 is not reachable from the host.

```
zsh — lab7 — 120x40

lab7 % curl -s localhost:3204/gateway-health; echo
{"status":"ok","service":"lab7-gateway"}
lab7 % curl -s localhost:3204/health; echo
{"status":"ok","service":"lab7-location-api"}
lab7 % curl -s 'localhost:3204/IP/search/?lat=0.37&long=0.63&cat=hospital&rad=0.3'; echo
["3562","3565","3660","3659","3966","3758","3365","3163","3467","4362"]
lab7 % curl -s -X POST localhost:3204/IP/search/ -F lat=0.5 -F long=0.5 -F cat=bank -F rad=0.8 -F link=@link.txt -w
'\nHTTP %{http_code} in %{time_total}s\n'
["4850","4948","4947","4648","4753","5349","4451","5348","4654","5347"]
HTTP 200 in 0.122408s
lab7 % curl -s 'localhost:3204/IP/search/?lat=0.5&long=0.5&cat=bank&rad=inf' -w '\nHTTP %{http_code}\n'
{"detail":"lat, long, and rad must be finite"}
HTTP 422
lab7 % curl -s 'localhost:3204/IP/search/?lat=0.5&long=0.5&cat=bank&rad=-1' -o /dev/null -w 'negative radius -> HTTP
%{http_code}\n'
negative radius -> HTTP 422
lab7 % curl -s localhost:8100/health -m 3 || echo 'port 8100 not published to host (API only reachable via gateway)'
port 8100 not published to host (API only reachable via gateway)
```

Figure 7: Functional tests through the gateway: health, GET search, POST with linkage file, validation errors.

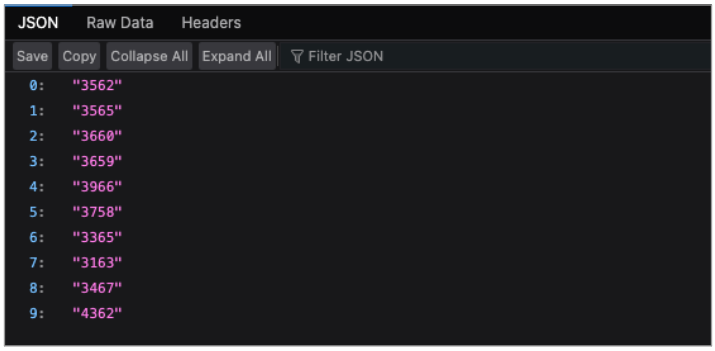


Figure 8: Firefox: http://localhost:3204/IP/search/?lat=0.37&long=0.63&cat=hospital&rad=0.3



Figure 9: Firefox: rad=inf rejected with HTTP 422 by the containerised API.

4.5 Load balancing and scaling

```
zsh — lab7 — 120x40

lab7 % for i in $(seq 1 20); do curl -s -o /dev/null -D - localhost:3204/health | grep X-Upstream; done | sort | uniq
-c
  10 X-Upstream: 172.19.0.2:8100
  10 X-Upstream: 172.19.0.3:8100
lab7 % docker compose up -d --scale api=4 --no-recreate
 Container lab7-gateway-1 Running
 Container lab7-api-1 Running
 Container lab7-api-2 Running
 Container lab7-api-4 Creating
 Container lab7-api-3 Creating
 Container lab7-api-3 Created
 Container lab7-api-4 Created
 Container lab7-api-4 Starting
 Container lab7-api-4 Started
 Container lab7-api-3 Starting
 Container lab7-api-3 Started
 Container lab7-api-4 Waiting
 Container lab7-api-3 Waiting
 Container lab7-api-1 Waiting
 Container lab7-api-2 Waiting
 Container lab7-api-4 Healthy
 Container lab7-api-3 Healthy
 Container lab7-api-1 Healthy
 Container lab7-api-2 Healthy
lab7 % sleep 15; docker compose ps --format 'table {{.Name}}\t{{.Status}}'
NAME             STATUS
lab7-api-1       Up 49 seconds (healthy)
lab7-api-2       Up 48 seconds (healthy)
lab7-api-3       Up 21 seconds (healthy)
lab7-api-4       Up 21 seconds (healthy)
lab7-gateway-1   Up 43 seconds (healthy)
lab7 % for i in $(seq 1 40); do curl -s -o /dev/null -D - localhost:3204/health | grep X-Upstream; done | sort | uniq
-c
  10 X-Upstream: 172.19.0.2:8100
  10 X-Upstream: 172.19.0.3:8100
  10 X-Upstream: 172.19.0.5:8100
  10 X-Upstream: 172.19.0.6:8100
```

Figure 10: The X-Upstream header shows an even round-robin split over 2 replicas, then over 4 after docker compose up --scale api=4.

```
zsh — lab7 — 120x40

lab7 % seq 1 400 | xargs -P 20 -I{} curl -s -o /dev/null -w '%{http_code}\n'
'localhost:3204/IP/search/?lat=0.37&long=0.63&cat=hospital&rad=0.3' | sort | uniq -c
 400 200
lab7 % docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.PIDs}}'
NAME             CPU %     MEM USAGE / LIMIT     PIDS
lab7-api-4       24.00%    119.6MiB / 512MiB     14
lab7-api-3       13.35%    128.9MiB / 512MiB     12
lab7-gateway-1   0.00%     7.969MiB / 3.826GiB   9
lab7-api-1       0.52%     126.5MiB / 512MiB     11
lab7-api-2       0.52%     118.6MiB / 512MiB     10
```

Figure 11: 400 requests at 20-way concurrency through the gateway: all HTTP 200. docker stats shows about 120 MiB per replica, within the 512 MiB limit.

4.6 Crash recovery

To test the restart policy, I sent 200 requests while making the Uvicorn master in `lab7-api-1` exit. Docker restarted it automatically (RestartCount 0 -> 1, new PID). During the outage the gateway retried requests on the other replica (`proxy_next_upstream`), so all 200 requests succeeded.



Figure 12: Crash under live traffic: 200/200 successful requests, container restarted and healthy again.

4.7 Isolation, volumes and the test suite

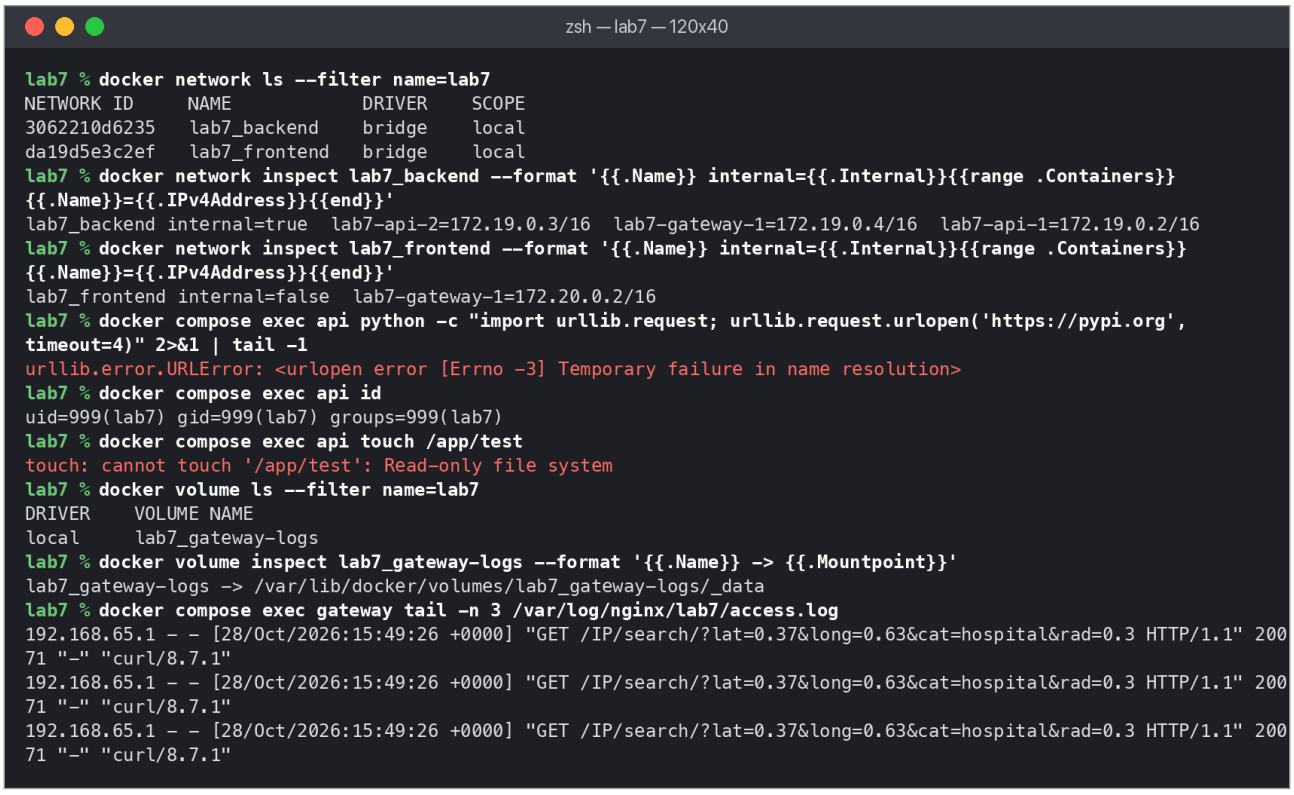


Figure 13: The backend network is internal: the API cannot resolve pypi.org. The API runs as uid 999 on a read-only filesystem, and the named volume holds the access log.

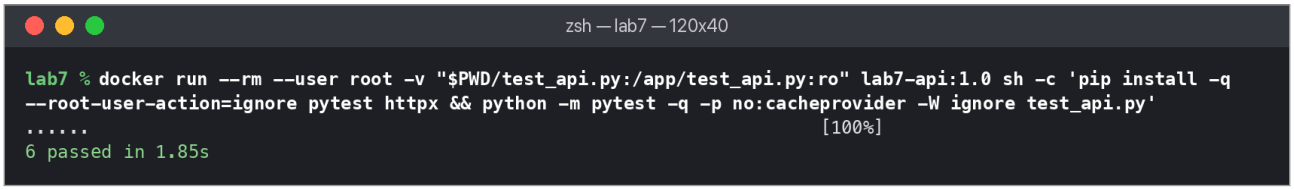


Figure 14: The existing Lab 7 test suite (6 tests, including a randomised Dijkstra check) passes inside the built image.

5. Errors faced and how I resolved them

These are the errors I actually hit while doing this lab, in the order they happened.

#	Error	Cause	Resolution
1	`failed to fetch anonymous token ... auth.docker.io ... connection reset by peer` on the first build	Unstable network connection to Docker Hub's token server. The image pull is the first thing the build does, so it failed straight away.	Retried with explicit `docker pull python:3.11-slim` and `docker pull nginx:1.28-alpine`. Once the base images were cached locally, later builds did not need to download them again (Fig. 15-16).
2	pip `SSLEOFError ... Retrying`; the pip step took 113.8 s and the whole build 1 m 57 s	Same unstable network, this time to PyPI. The original Dockerfile used `--no-cache-dir`, so any rebuild would download everything again.	Added `PIP_DEFAULT_TIMEOUT=60` and `--retries 10`, and replaced --no-cache-dir with a BuildKit cache mount (`--mount=type=cache,target=/root/.cache/pip`) that never ends up in the image. A full rebuild now takes 37 s; a cached one takes 6.7 s.
3	`# syntax=docker/dockerfile:1.7` failed: `resolve image config for docker-image://docker.io/docker/dockerfile:1.7 ... connection reset` (only the gateway image was built)	The syntax line makes BuildKit download a separate frontend image from Docker Hub, which added another network dependency.	Removed the line. The BuildKit frontend built into Docker 29 already supports `RUN --mount`, so nothing else changed (Fig. 17).
4	The baseline image ran as `uid=0(root)`, copied `app/__pycache__` into the image, and pip warned about running as root	The original Dockerfile had no USER instruction and no .dockerignore.	Added a system user `lab7` with `USER lab7`, a .dockerignore (excluding caches, docx/pdf, link.txt and tests), and `PIP_ROOT_USER_ACTION=ignore` for the build-time install (Fig. 18).
5	`exec: "kill": executable file not found in $PATH` when running `docker exec lab7-api-1 kill -TERM 1`	`kill` is a shell builtin, and `docker exec` runs a binary directly. The slim image does not include procps.	Sent the signal with the Python already in the image: `python -c 'import os, signal; os.kill(1, signal.SIGTERM)'`.
6	After `docker kill lab7-api-1` the container stayed `Exited (137)` with RestartCount=0	Not a bug: `unless-stopped` deliberately does not restart a container that was stopped or killed by the user. My test was simulating an operator action, not a crash.	Brought it back with `docker compose start api`, then simulated a real crash from inside the container (error 5 fix). Docker restarted it (RestartCount=1) and no requests failed, because the gateway failed over to the other replica (Fig. 19, Fig. 12).
7	Replicas added with `--scale api=4` got no traffic from a plain Nginx `upstream { server api:8100; }`	Open-source Nginx normally resolves upstream hostnames once, at startup, so new replica IPs are never seen. I tested this with a temporary plain-config gateway: after scaling to 4 it still used only the original 2 IPs.	Used `resolver 127.0.0.11 valid=10s` with `zone` and `server api:8100 resolve` in the upstream. The real gateway spread 40 requests evenly over all 4 replicas (Fig. 20).

```
zsh — lab7 — 120x40

lab7 % docker build --progress=plain -t lab7-api:naive .
#0 building with "desktop-linux" instance using docker driver
#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 328B 0.0s done
#1 DONE 0.0s
#2 [internal] load metadata for docker.io/library/python:3.11-slim
#2 ERROR: failed to authorize: failed to fetch anonymous token: Get
"https://auth.docker.io/token?scope=repository%3Alibrary%2Fpython%3Apull&service=registry.docker.io": read tcp
10.50.29.178:55656->172.64.144.78:443: read: connection reset by peer
------
 > [internal] load metadata for docker.io/library/python:3.11-slim:
------
Dockerfile:1
--------------------
   1 | >>> FROM python:3.11-slim
   2 |
   3 |     WORKDIR /app
--------------------
ERROR: failed to build: failed to solve: failed to fetch anonymous token: Get
"https://auth.docker.io/token?scope=repository%3Alibrary%2Fpython%3Apull&service=registry.docker.io": read tcp
10.50.29.178:55656->172.64.144.78:443: read: connection reset by peer
```

Figure 15: Error 1: Docker Hub token request reset during the first build.

```
zsh — lab7 — 120x40

lab7 % docker pull python:3.11-slim
2bdd609874cf: Pull complete
Digest: sha256:673719e071ecf27434fd2635bbd8d00f49f4fdd81f243143438a42ced8bedf60
Status: Downloaded newer image for python:3.11-slim
docker.io/library/python:3.11-slim
lab7 % docker pull nginx:1.28-alpine
Digest: sha256:a8b39bd9cf0f83869a2162827a0caf6137ddf759d50a171451b335cecc87d236
Status: Downloaded newer image for nginx:1.28-alpine
docker.io/library/nginx:1.28-alpine
```

Figure 16: Error 1 fix: base images pulled explicitly and cached locally.

```
zsh — lab7 — 120x40

lab7 % docker compose build
#8 [api] resolve image config for docker-image://docker.io/docker/dockerfile:1.7
#8 ERROR: failed to authorize: failed to fetch anonymous token: Get
"https://auth.docker.io/token?scope=repository%3Adocker%2Fdockerfile%3Apull&service=registry.docker.io": read tcp
10.50.29.178:55711->172.64.144.78:443: read: connection reset by peer
------
 > [api] resolve image config for docker-image://docker.io/docker/dockerfile:1.7:
------
Dockerfile:1
--------------------
   1 | >>> # syntax=docker/dockerfile:1.7
   2 |     # Lab 7 proximity search API (FastAPI + Uvicorn)
   3 |     FROM python:3.11-slim
--------------------
target api: failed to solve: failed to fetch anonymous token: Get
"https://auth.docker.io/token?scope=repository%3Adocker%2Fdockerfile%3Apull&service=registry.docker.io": read tcp
10.50.29.178:55711->172.64.144.78:443: read: connection reset by peer
```

Figure 17: Error 3: the # syntax directive tried to download docker/dockerfile:1.7 and failed.

```
zsh — lab7 — 120x40

lab7 % time docker build --progress=plain -t lab7-api:naive .
#5 transferring context: 327.33kB 0.0s done
#8 [4/6] RUN pip install --no-cache-dir -r requirements.txt
#8 1.756 WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection
broken by 'SSLError(SSLEOFError(8, '[SSL: UNEXPECTED_EOF_WHILE_READING] EOF occurred in violation of protocol
(_ssl.c:1018)'))': /simple/fastapi/
#8 112.6 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the
system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv
#8 DONE 113.8s
docker build --progress=plain -t lab7-api:naive .  0.25s user 0.31s system 0% cpu 1:57.00 total
lab7 % docker run --rm lab7-api:naive sh -c 'ls -la /app; id'
total 316
drwxr-xr-x 1 root root   4096 Oct 28 15:40 .
drwxr-xr-x 1 root root   4096 Oct 28 15:40 ..
drwxr-xr-x 3 root root   4096 Oct  7 22:21 app
-rw-r--r-- 1 root root 305774 Oct  7 22:21 locations.csv
-rw-r--r-- 1 root root     72 Oct  7 22:21 requirements.txt
uid=0(root) gid=0(root) groups=0(root)
lab7 % docker image ls lab7-api
IMAGE            ID             DISK USAGE   CONTENT SIZE   EXTRA
lab7-api:naive   9360954f2f0d        200MB             0B
```

Figure 18: Errors 2 and 4: pip SSL retry, 1 m 57 s build, root user and root pip warning in the baseline image.

```
zsh — lab7 — 120x40

lab7 % docker exec lab7-api-1 kill -TERM 1
OCI runtime exec failed: exec failed: unable to start container process: exec: "kill": executable file not found in
$PATH
lab7 % docker kill lab7-api-1
lab7-api-1
lab7 % sleep 15; docker compose ps -a --format 'table {{.Name}}\t{{.Status}}'
NAME             STATUS
lab7-api-1       Exited (137) 15 seconds ago
lab7-api-2       Up About a minute (healthy)
lab7-gateway-1   Up About a minute (healthy)
lab7 % docker inspect lab7-api-1 --format 'RestartCount={{.RestartCount}} ExitCode={{.State.ExitCode}}'
RestartCount=0 ExitCode=137
```

Figure 19: Errors 5 and 6: kill is not present in the slim image; docker kill is a manual stop, so it is not restarted.

```
zsh — lab7 — 120x40

lab7 % cat plain/default.conf
upstream lab7_api { server api:8100; }
server { listen 3204; location / { proxy_pass http://lab7_api; add_header X-Upstream $upstream_addr always; } }
lab7 % docker run -d --name plain-gw --network lab7_backend -v
$PWD/plain/default.conf:/etc/nginx/conf.d/default.conf:ro nginx:1.28-alpine
lab7 % # 2 replicas running
lab7 % for i in $(seq 1 20); do docker exec plain-gw wget -S -q -O /dev/null http://127.0.0.1:3204/health 2>&1 | grep
X-Upstream; done | sort | uniq -c
   6   X-Upstream: 172.19.0.2:8100
  14   X-Upstream: 172.19.0.3:8100
lab7 % docker compose up -d --scale api=4 --no-recreate
 Container lab7-api-1 Healthy
lab7 % for i in $(seq 1 40); do docker exec plain-gw wget -S -q -O /dev/null http://127.0.0.1:3204/health 2>&1 | grep
X-Upstream; done | sort | uniq -c
  20   X-Upstream: 172.19.0.2:8100
  20   X-Upstream: 172.19.0.3:8100
lab7 % # same test against the real gateway (upstream uses `resolve`)
lab7 % for i in $(seq 1 40); do curl -s -o /dev/null -D - localhost:3204/health | grep X-Upstream; done | sort | uniq
-c
  10 X-Upstream: 172.19.0.2:8100
  10 X-Upstream: 172.19.0.3:8100
  10 X-Upstream: 172.19.0.6:8100
  10 X-Upstream: 172.19.0.7:8100
```

Figure 20: Error 7: a plain upstream ignores new replicas, while the real gateway (resolve) uses all four.

6. Docker commands used

6.1 Commands in the order I ran them

```
# install check
docker --version
docker compose version
docker info --format '...'
# baseline build of the original Dockerfile
docker build --progress=plain -t lab7-api:naive .          # error 1
docker pull python:3.11-slim
docker pull nginx:1.28-alpine
docker build --progress=plain -t lab7-api:naive .          # error 2, 4
docker image ls lab7-api
docker run --rm lab7-api:naive sh -c 'ls -la /app; id'
# final setup
cp .env.example .env
docker compose config
docker compose build                                       # error 3
docker compose build --progress=plain
docker compose up -d --build
docker compose ps
docker compose logs --no-color
# testing
curl -s localhost:3204/health   (and the other curl tests in Fig. 7)
docker compose exec api id
docker compose exec api touch /app/test
docker compose exec api python -c "...urlopen('https://pypi.org')..."
docker compose exec gateway wget -q -O- http://api:8100/health
docker compose exec gateway tail -n 3 /var/log/nginx/lab7/access.log
docker network ls --filter name=lab7
docker network inspect lab7_backend --format '...'
docker volume ls --filter name=lab7
docker volume inspect lab7_gateway-logs --format '...'
docker image ls --filter reference='lab7-*'
docker compose images
# resilience
docker exec lab7-api-1 kill -TERM 1                        # error 5
docker kill lab7-api-1                                     # error 6
docker compose ps -a
docker inspect lab7-api-1 --format '{{.RestartCount}} ...'
docker compose start api
docker exec lab7-api-1 python -c 'import os, signal; os.kill(1, signal.SIGTERM)'
# scaling and load
docker compose up -d --scale api=4 --no-recreate
docker stats --no-stream --format 'table ...'
docker run -d --name plain-gw --network lab7_backend     -v ...:/etc/nginx/conf.d/default.conf:ro nginx:1.28-alpine
docker exec plain-gw wget -S -q -O /dev/null http://127.0.0.1:3204/health
docker rm -f plain-gw
docker compose up -d --scale api=2
# test suite inside the image
docker run --rm --user root -v "$PWD/test_api.py:/app/test_api.py:ro" lab7-api:1.0     sh -c 'pip install pytest httpx &&
# teardown / clean restart
docker compose down
docker compose down -v --rmi local
docker image rm lab7-api:naive
```

6.2 Unique commands and their purpose

Command	Purpose
`docker --version / docker compose version`	Show CLI and Compose plugin versions
`docker info`	Show engine details (version, OS, architecture, CPUs, memory)
`docker pull IMAGE`	Download an image from a registry into the local cache
`docker build -t NAME .`	Build an image from a Dockerfile and its build context
`docker image ls`	List local images and their sizes
`docker image rm IMAGE`	Delete a local image
`docker run [--rm] [-d] [-v] [--network] IMAGE CMD`	Create and start a new container (one-off or detached)
`docker exec CONTAINER CMD`	Run a command inside a running container

Command	Purpose
`docker kill CONTAINER`	Send SIGKILL to a container (counts as a manual stop)
`docker inspect CONTAINER --format`	Read low-level container state (PID, RestartCount, health)
`docker stats --no-stream`	One-time snapshot of CPU, memory and PID usage per container
`docker rm -f CONTAINER`	Force-remove a container
`docker network ls / inspect`	List networks and show their members, IPs and internal flag
`docker volume ls / inspect`	List volumes and show where a volume is stored
`docker compose config`	Validate the compose file and print it with variables filled in
`docker compose build`	Build every service image defined in the compose file
`docker compose up -d --build`	The one command: build, create networks and volumes, start everything in the background
`docker compose up -d --scale api=N`	Change the number of replicas of a service
`docker compose ps [-a]`	Show service containers, their status, health and ports
`docker compose images`	Show the images used by the running services
`docker compose logs`	Show combined logs from all services
`docker compose exec SERVICE CMD`	Run a command inside a running service container
`docker compose start SERVICE`	Start existing stopped containers of a service
`docker compose down [-v] [--rmi local]`	Stop and remove containers and networks (-v also removes volumes)

7. What I learnt

- Layer order is what makes builds fast. Installing requirements before copying the code, and using a BuildKit cache mount, brought a 2-minute build down to seconds when only code changes. A .dockerignore matters just as much for what ends up in the image as for build speed.
- Compose replaces most of what I set up by hand on the cluster. `restart: unless-stopped` does the job of Supervisor's autorestart, replicas replace the four sys nodes, and the service name replaces hard-coded IPs and ports.
- **Health checks plus** `depends_on: condition: service_healthy` fix startup order properly. Plain `depends_on` only waits for the container to start, not for the CSV to load and the API to answer.
- Docker networking: containers on a user-defined network find each other by name through the embedded DNS at 127.0.0.11. Marking a network `internal` is an easy way to keep a backend off the internet, and only the gateway needs a published port.
- **Restart policies follow intent.** `unless-stopped` restarts a container that crashed but not one the user killed. Testing recovery properly meant making the process fail from inside the container.
- PID 1 matters. With `exec`, Uvicorn receives SIGTERM directly and shuts down cleanly. Minimal images also leave out tools I took for granted (curl, kill, ps), so health checks and debugging have to use what is already in the image.
- A reverse proxy in front of replicas needs to re-resolve DNS, otherwise scaling does nothing. Adding an `X-Upstream` header made the load balancing easy to see and check.
- The network on a personal laptop is part of the build environment. Retries, timeouts and pre-pulled base images make builds much more reliable than assuming downloads always succeed.

8. Conclusion

The Lab 7 Proximity Search API now runs as a two-service Docker stack, an Nginx gateway and a scalable API service, started with `docker compose up -d --build`. It returns the same results as the cluster deployment, passes the existing test suite inside the image, balances load across replicas, recovers from a process crash with no failed requests, and keeps the API off the public network.

Appendix: Docker files

A. lab7/Dockerfile (api service)

```
# Lab 7 proximity search API (FastAPI + Uvicorn)
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    PIP_DEFAULT_TIMEOUT=60 \
    PIP_ROOT_USER_ACTION=ignore

WORKDIR /app

# Dependencies first so code edits do not reinstall packages.
# The pip cache mount survives between builds but never ships in the image.
RUN --mount=type=cache,target=/root/.cache/pip \
    --mount=type=bind,source=requirements.txt,target=/tmp/requirements.txt \
    pip install --retries 10 -r /tmp/requirements.txt

RUN groupadd --system lab7 && useradd --system --gid lab7 --no-create-home lab7

COPY app ./app
# Default dataset baked in; docker-compose can bind-mount a different CSV over it.
COPY locations.csv /data/locations.csv

ENV LOCATION_DATASET=/data/locations.csv \
    API_PORT=8100 \
    UVICORN_WORKERS=2 \
    UVICORN_LIMIT_CONCURRENCY=64 \
    UVICORN_BACKLOG=2048

USER lab7
EXPOSE 8100

HEALTHCHECK --interval=10s --timeout=3s --start-period=20s --retries=3 \
    CMD ["python", "-c", "import os, urllib.request; urllib.request.urlopen('http://127.0.0.1:' + \
    os.environ.get('API_PORT', '8100') + '/health', timeout=2)"]

# exec makes Uvicorn PID 1 so it receives SIGTERM from `docker stop`.
CMD ["sh", "-c", "exec uvicorn app.main:app --host 0.0.0.0 --port \"$API_PORT\" --workers \"$UVICORN_WORKERS\" \
    --limit-concurrency \"$UVICORN_LIMIT_CONCURRENCY\" --backlog \"$UVICORN_BACKLOG\" --timeout-keep-alive 5 \
    --proxy-headers --forwarded-allow-ips '*'"]
```

B. lab7/gateway/Dockerfile (gateway service)

```
# Lab 7 gateway: Nginx reverse proxy and load balancer in front of the API replicas
FROM nginx:1.28-alpine

# The stock image renders /etc/nginx/templates/*.template into conf.d/ with
# envsubst at startup, so API_HOST, API_PORT and GATEWAY_PORT are set at run time.
RUN rm /etc/nginx/conf.d/default.conf && mkdir -p /var/log/nginx/lab7
COPY templates/ /etc/nginx/templates/

ENV API_HOST=api \
    API_PORT=8100 \
    GATEWAY_PORT=3204

EXPOSE 3204

HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
    CMD wget -q -O /dev/null "http://127.0.0.1:${GATEWAY_PORT}/gateway-health" || exit 1
```

C. lab7/gateway/templates/default.conf.template

```
# Docker's embedded DNS. Re-resolving every 10 s lets Nginx follow API
# replicas as they are scaled, restarted or given new IP addresses.
resolver 127.0.0.11 valid=10s ipv6=off;

upstream lab7_api {
    zone lab7_api 64k;
    server ${API_HOST}:${API_PORT} resolve;
    keepalive 16;
}

server {
    listen ${GATEWAY_PORT};
    server_name _;

    # The API rejects linkage files above 64 MiB with HTTP 413 itself.
    client_max_body_size 65m;

    access_log /dev/stdout;
    access_log /var/log/nginx/lab7/access.log;

    location = /gateway-health {
        access_log off;
        default_type application/json;
        return 200 '{"status":"ok","service":"lab7-gateway"}';
    }

    location / {
        proxy_pass http://lab7_api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 5s;
        proxy_read_timeout 60s;
        proxy_next_upstream error timeout http_502 http_503;
        add_header X-Upstream $upstream_addr always;
    }
}
```

D. lab7/docker-compose.yml

```
# Lab 7 proximity search stack. Start everything with:
#   docker compose up -d --build
name: lab7

services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
    image: lab7-api:1.0
    environment:
      LOCATION_DATASET: /data/locations.csv
      API_PORT: ${API_PORT:-8100}
      UVICORN_WORKERS: ${UVICORN_WORKERS:-2}
      UVICORN_LIMIT_CONCURRENCY: ${UVICORN_LIMIT_CONCURRENCY:-64}
      UVICORN_BACKLOG: ${UVICORN_BACKLOG:-2048}
    volumes:
      # Swap in another dataset (e.g. a 1M-row CSV) without rebuilding.
      - type: bind
        source: ${DATASET_FILE:-./locations.csv}
        target: /data/locations.csv
        read_only: true
    expose:
      - "${API_PORT:-8100}"
    networks:
      - backend
    read_only: true
    tmpfs:
      - /tmp
    deploy:
      replicas: ${API_REPLICAS:-2}
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
    restart: unless-stopped
    logging: &default-logging
      driver: json-file
      options:
        max-size: "10m"
        max-file: "5"

  gateway:
    build:
      context: ./gateway
    image: lab7-gateway:1.0
    environment:
      API_HOST: api
      API_PORT: ${API_PORT:-8100}
      GATEWAY_PORT: "3204"
    ports:
      - "${GATEWAY_PORT:-3204}:3204"
    volumes:
      - gateway-logs:/var/log/nginx/lab7
    networks:
      - frontend
      - backend
    depends_on:
      api:
        condition: service_healthy
    restart: unless-stopped
    logging: *default-logging

networks:
  # Published gateway port lives here.
  frontend:
  # No route to the host or internet; only the gateway can reach the API.
  backend:
    internal: true

volumes:
  gateway-logs:
```

E. lab7/.dockerignore

```
# Only app/, requirements.txt and locations.csv belong in the API image.
**/__pycache__/
**/*.py[cod]
.pytest_cache/
test_api.py
*.md
*.docx
*.pdf
link.txt
lab7-supervisor.conf
docker-compose.yml
.env
.env.example
gateway/
.DS_Store
```

F. lab7/.env.example

```
# Copy to .env to override the defaults in docker-compose.yml.
GATEWAY_PORT=3204
API_PORT=8100
API_REPLICAS=2
UVICORN_WORKERS=2
UVICORN_LIMIT_CONCURRENCY=64
UVICORN_BACKLOG=2048
DATASET_FILE=./locations.csv
```